Cláusulas Avançadas em SQL (CREATE)

Linguagem de Definição de Dados (DDL)

- **Criar uma Table (Tabela):**
  - Dentro de um schema, você pode criar tabelas para definir a estrutura onde os dados serão armazenados:

Sintaxe

```
CREATE TABLE nome_do_esquema.nome_da_tabela (
    coluna1 tipo_de_dado1 CONSTRAINTS,
    coluna2 tipo_de_dado2 CONSTRAINTS,
    coluna3 tipo_de_dado3 CONSTRAINTS,
    ...
);
```

Sintaxe

```
CREATE TABLE clientes.informacoes (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    data_de_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Linguagem de Definição de Dados (DDL)

- Frequentemente criamos **relações** entre tabelas usando chaves estrangeiras (**FOREIGN KEY**).

- Essas relações são essenciais para **garantir a integridade referencial**, ou seja, assegurar que os dados relacionados permaneçam consistentes entre as tabelas.

- Há situações em que devemos considerar o que acontece quando os valores das chaves primárias (referenciadas por outras tabelas) mudam.

O que são ON DELETE e ON UPDATE?

- As cláusulas **ON DELETE** e **ON UPDATE** são usadas em conjunto com chaves estrangeiras (**FOREIGN KEY**) em SQL.

- Elas definem o comportamento do banco de dados quando a linha referenciada por uma chave estrangeira é deletada (**ON DELETE**) ou atualizada (**ON UPDATE**).

- Essas cláusulas ajudam a manter a **integridade referencial** e garantem que as relações entre tabelas permaneçam consistentes.

Quando usar ON DELETE e ON UPDATE?

- **ON DELETE:** define o que acontece com os registros dependentes (na “tabela filha”) quando uma linha da “tabela pai” (referenciada pela chave primária/única) é excluída.

- **ON UPDATE:** define o que acontece com os registros dependentes (na tabela filha) quando **atualizar o valor da chave primária na “tabela pai”**

Sintaxe Básica

comandos.sql

```sql
CREATE TABLE nome_da_tabela (
    coluna1 tipo_de_dado CONSTRAINTS,
    coluna2 tipo_de_dado CONSTRAINTS,
    ...
    FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
    ON DELETE ação
    ON UPDATE ação
);
```

Possibilidades para ON DELETE e ON UPDATE

➔ CASCADE

- **Descrição:** Quando uma linha referenciada é deletada ou atualizada, todas as linhas na tabela que possuem a chave estrangeira correspondente são deletadas ou atualizadas automaticamente.

- **Quando usar:** Use CASCADE quando a existência das linhas dependentes (referenciadas pela chave estrangeira) não fizer sentido sem a linha pai.

comandos.sql

```
FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
ON DELETE CASCADE
ON UPDATE CASCADE
```

Possibilidades para ON DELETE e ON UPDATE

➔ SET NULL

- **Descrição:** Quando uma linha referenciada é deletada ou atualizada, as colunas de chave estrangeira nas linhas dependentes são definidas como NULL.

- **Quando usar:** Use SET NULL quando quiser manter as linhas dependentes, mas remover a referência à linha pai.

comandos.sql

```sql
FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
ON DELETE SET NULL
ON UPDATE SET NULL
```

Possibilidades para ON DELETE e ON UPDATE

➔ SET DEFAULT

- **Descrição:** Quando uma linha referenciada é deletada ou atualizada, as colunas de chave estrangeira nas linhas dependentes são definidas com um valor padrão especificado.

- **Quando usar:** Use SET DEFAULT quando quiser que as linhas dependentes assumam um valor padrão que indica uma condição especial.

comandos.sql

```
FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
ON DELETE SET DEFAULT
ON UPDATE SET DEFAULT

```

Possibilidades para ON DELETE e ON UPDATE

➔ RESTRICT

- **Descrição:** Impede a deleção ou atualização de uma linha referenciada se existem linhas dependentes que fazem referência a ela.
- **Quando usar:** Use RESTRICT para evitar a perda acidental de integridade referencial.

comandos.sql

```sql
FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
ON DELETE RESTRICT
ON UPDATE RESTRICT
```

Possibilidades para ON DELETE e ON UPDATE

➔ NO ACTION

- **Descrição:** Similar a RESTRICT, mas a verificação é diferida até o final da transação. É o comportamento padrão se nenhuma ação for especificada.

- **Quando usar:** Use NO ACTION quando quiser que as verificações de integridade referencial ocorram após a execução de outras operações

comandos.sql

```
FOREIGN KEY (coluna_estrangeira) REFERENCES tabela_referenciada(coluna_referenciada)
ON DELETE NO ACTION
ON UPDATE NO ACTION
```

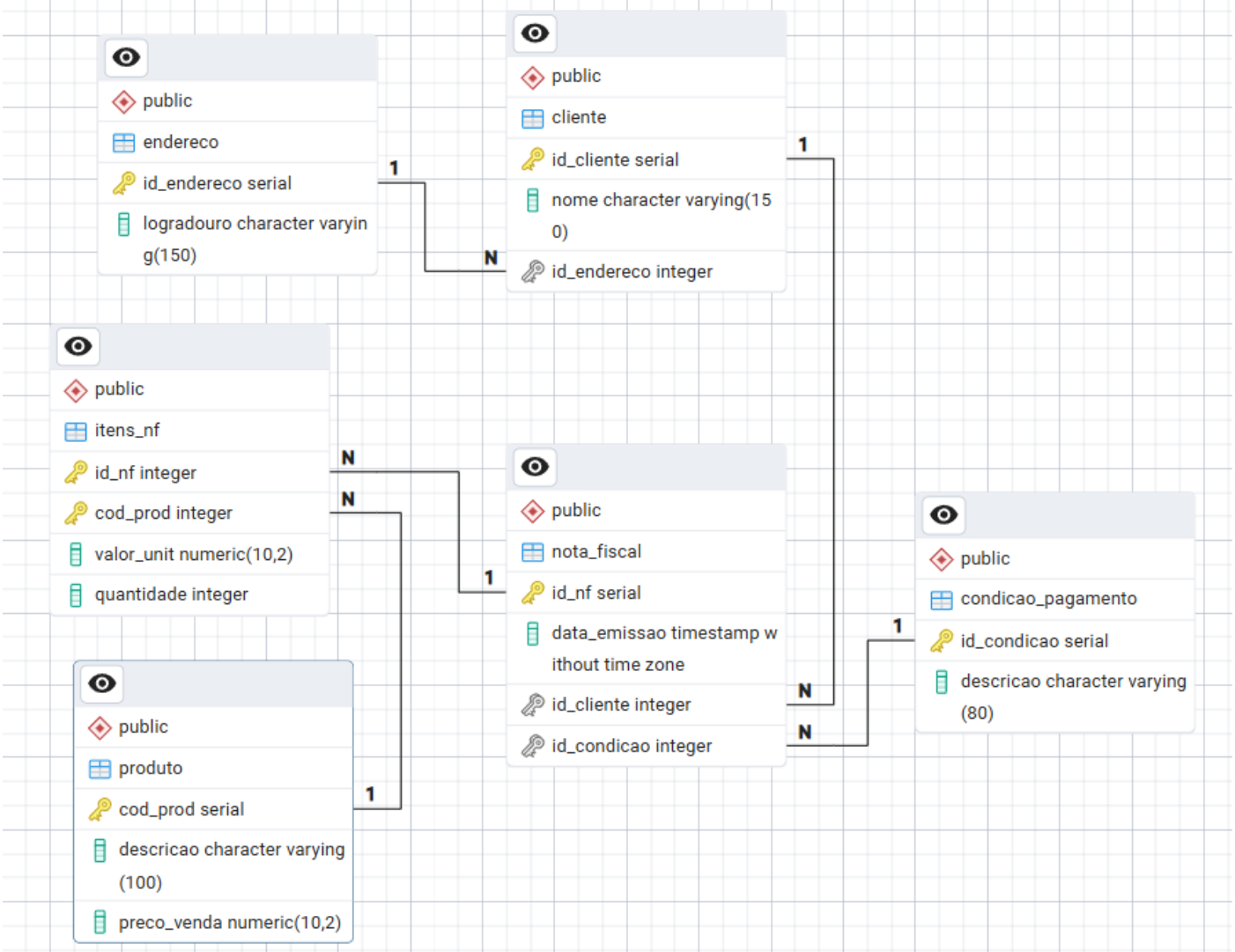
Possibilidades para ON DELETE e ON UPDATE

➔ NO ACTION

- No **PostgreSQL** (e na maioria dos SGBDs relacionais), quando você **não especifica nada** no ON DELETE ou ON UPDATE o **padrão** é NO ACTION

- **RESTRICT** → A verificação é **imediata**: o SGBD barra a operação na hora que encontra o conflito.

- **NO ACTION** → A verificação é **adiada** até o final da instrução ou da transação.


public
endereco
id_endereco serial
logradouro character varyin
g(150)
1
N
public
cliente
id_cliente serial
nome character varying(15
0)
id_endereco integer
1
public
itens_nf
id_nf integer
cod_prod integer
valor_unit numeric(10,2)
quantidade integer
N
N
1
public
nota_fiscal
id_nf serial
data_emissao timestamp w
ithout time zone
id_cliente integer
id_condicao integer
N
N
1
public
condicao_pagamento
id_condicao serial
descricao character varying
(80)
1
public
produto
cod_prod serial
descricao character varying
(100)
preco_venda numeric(10,2)

Cláusulas Avançadas em SQL utilizadas com SELECT

Cláusulas

- HAVING
- ORDER BY
- LIMIT e OFFSET
- DISTINCT
- UNION, UNION ALL
- WINDOW FUNCTIONS

HAVING

- A cláusula HAVING é usada para filtrar grupos de linhas criados pela cláusula GROUP BY.

- Funciona de maneira semelhante à cláusula WHERE, mas é aplicada após a agregação.

```sql
SELECT
    i.id_nf,
    SUM(i.valor_unit * i.quantidade) AS total_nf
FROM itens_nf i
GROUP BY i.id_nf
HAVING SUM(i.valor_unit * i.quantidade) > 20;
```

ORDER BY

- A cláusula ORDER BY é usada para ordenar os resultados de uma consulta em ordem crescente (ASC) ou decrescente (DESC).

```sql
SELECT
    id_cliente,
    nome,
    id_endereco
FROM cliente
ORDER BY nome ASC;
```

LIMIT

- A cláusula LIMIT é usada para limitar o número de linhas retornadas pela consulta.

```sql
SELECT
    id_cliente,
    nome
FROM cliente
ORDER BY nome ASC
LIMIT 5;
```

LIMIT e OFFSET

- A cláusula OFFSET é usada para pular um número específico de linhas antes de começar a retornar as linhas da consulta, usada junto com o limit cria paginação

- Esse comando retorna apenas 5 registros, começando a partir do 6º cliente.

```sql
SELECT
    id_cliente,
    nome
FROM cliente
ORDER BY nome ASC
LIMIT 5 OFFSET 5;
```

DISTINCT

- A cláusula DISTINCT é usada para remover duplicatas das linhas retornadas pela consulta.

```
SELECT DISTINCT id_endereco
FROM cliente;
```

UNION e UNION ALL

- A cláusula UNION é usada para combinar os resultados de duas ou mais consultas SELECT, removendo duplicatas.

- A cláusula UNION ALL faz o mesmo, mas mantém todas as duplicatas.

comandos.sql

```sql
SELECT nome FROM Alunos
UNION
SELECT nome FROM Professores;

SELECT nome FROM Alunos
UNION ALL
SELECT nome FROM Professores;
```

```sql
SELECT nome AS dado
FROM cliente
UNION
SELECT logradouro
FROM endereco;
```

WINDOW FUNCTIONS

- Funções de janela (window functions) permitem que você realize cálculos sobre um conjunto de linhas relacionadas à linha atual. São usadas com a cláusula OVER.

- Diferem das funções de agregação normais (SUM, AVG, etc.) porque não colapsam o resultado em uma única linha por grupo; elas mantêm cada linha original, mas adicionam uma coluna calculada.

WINDOW FUNCTIONS

- O exemplo abaixo calcula a idade média dos alunos para cada curso (curso_id) e adiciona essa média como uma coluna avg_idade na tabela de resultados.

comandos.sql

```sql
SELECT nome, curso_id,
       AVG(idade) OVER (PARTITION BY curso_id) AS avg_idade
FROM Alunos;
```

WINDOW FUNCTIONS

```sql
SELECT
    id_nf,
    cod_prod,
    (quantidade * valor_unit) AS valor_total,
    AVG(valor_unit * quantidade)
        OVER (PARTITION BY id_nf) AS media_nf
FROM itens_nf;
```

Views

O que são Views?

- Uma view é uma tabela virtual baseada no resultado de uma consulta SQL.

- Ela não armazena dados fisicamente, mas sim a consulta que gera os dados.

- As views permitem que você crie uma camada de abstração sobre suas tabelas, facilitando a consulta e a manipulação de dados.

Suponha as Seguintes Tabelas

comandos.sql

```sql
CREATE VIEW nome_da_view AS
SELECT colunas
FROM tabela
WHERE condição;
```

Suponha as seguintes Tabelas

alunos

id	nome	idade	curso_id
1	Ana	21	101
2	Bruno	22	102
3	Carlos	23	101
4	Daniela	20	103

cursos

id	nome
101	Engenharia de Software
102	Ciência da Computação
103	Administração

Criando uma View

comandos.sql

```sql
CREATE VIEW AlunosComCursos AS
SELECT Alunos.id, Alunos.nome, Alunos.idade, Cursos.nome AS curso_nome
FROM Alunos
INNER JOIN Cursos ON Alunos.curso_id = Cursos.id;
```

id	nome	idade	curso_nome
2	Bruno	22	Ciência da Computação
3	Carlos	23	Engenharia de Software
4	Daniela	20	Administração
1	Ana	21	Engenharia de Software

Continua na próxima aula...



UniRV
Universidade de Rio Verde

UNIVERSIDADE DE RIO VERDE (UniRV)

Bacharelado em Engenharia de Software

Fasoft
Faculdade de Engenharia de Software